

PREGUNTA 1

¿Por qué falla con 127.0.0.1?

127.0.0.1 (loopback) significa "escucha solo en conexiones que vengan del mismo proceso/máquina". Dentro del contenedor, eso limita uvicorn a conexiones originadas dentro del propio contenedor. El tráfico que llega desde fuera — incluido el mapeo de puertos de Docker (-p 8000:8000) — entra por la interfaz de red virtual del contenedor (eth0), no por loopback, por lo que uvicorn lo ignora.

¿Qué significa 0.0.0.0?

0.0.0.0 significa "escucha en todas las interfaces de red disponibles en este contenedor": loopback (127.0.0.1), la interfaz interna de Docker (eth0) y cualquier otra. Así uvicorn acepta conexiones venga de donde venga.

¿Por qué es necesario para el mapeo de puertos?

El mapeo -p 8000:8000 hace que Docker reenvíe el tráfico que llega al puerto 8000 del host hacia el puerto 8000 del contenedor, pero lo entrega a través de la interfaz eth0 del contenedor. Si uvicorn solo escucha en 127.0.0.1, ese tráfico llega a una interfaz que no está siendo escuchada y la conexión se rechaza. Con 0.0.0.0, uvicorn escucha en eth0 también, y el reenvío de Docker funciona.

PREGUNTA 2

¿Por qué localhost no resuelve al backend?

Cada contenedor tiene su propio stack de red aislado. localhost (o 127.0.0.1) dentro del contenedor frontend apunta al loopback del propio contenedor frontend, no al host ni a ningún otro contenedor. No hay ningún proceso escuchando en el puerto 8000 del frontend, por eso la conexión se rechaza.

¿Qué mecanismo usa Docker Compose para que backend sea un nombre DNS válido?

Cuando Docker Compose levanta el stack, crea automáticamente una red bridge privada (en este caso iris-network) y conecta todos los servicios a ella. Docker incluye un servidor DNS embebido (en la IP 127.0.0.11 dentro de cada contenedor) que resuelve los nombres de servicio definidos en docker-compose.yml a la IP interna del contenedor correspondiente.

Es decir: cuando el frontend hace una petición a `http://backend:8000`, su resolver DNS consulta al DNS de Docker, que devuelve la IP interna del contenedor iris-backend en la red iris-network. La conexión viaja por esa red bridge interna sin salir al host.

Esto no funciona fuera de Compose (por eso al lanzar el frontend con `docker run` suelto hay que usar `host.docker.internal`) porque el contenedor no está conectado a la misma red ni tiene al DNS de Compose disponible.

PREGUNTA 3

Mecanismo de caché de capas de Docker

Cada instrucción en un Dockerfile genera una capa. Docker cachea cada capa y la reutiliza en builds posteriores siempre que ni esa instrucción ni ninguna capa anterior hayan cambiado. En cuanto detecta un cambio, invalida esa capa y todas las que vienen después, forzando su reconstrucción.

¿Qué ocurre si se invierte el orden?

Con `COPY . .` antes del `RUN pip install`:

```
COPY . .          # se invalida con cualquier cambio de código
RUN pip install -r requirements.txt # se re-ejecuta siempre
```

Cualquier modificación en cualquier fichero del proyecto (incluso un comentario en `main.py`) invalida el `COPY . .`, lo que arrastra la invalidación al `RUN pip install`. Docker reinstala todas las dependencias desde cero en cada build, aunque `requirements.txt` no haya cambiado.

¿Por qué el orden original es mejor?

```
COPY requirements.txt .          # solo se invalida si cambia requirements.txt
RUN pip install -r requirements.txt # se cachea mientras requirements.txt no cambie
COPY . .                          # se invalida con cualquier cambio de código
```

pip install es la operación más lenta del build. Al copiarlo por separado antes que el resto del código, esa capa solo se reconstruye cuando cambian las dependencias. Los cambios habituales de desarrollo (modificar main.py, gradio_app.py, etc.) reutilizan la capa cacheada del pip install y el build se reduce de minutos a segundos.

PREGUNTA 4

Riesgo principal de la solución .env

El fichero .env existe en texto plano en el disco del desarrollador. Aunque esté en .gitignore, el riesgo real es que se suba accidentalmente al repositorio, se copie en una imagen Docker (COPY . . sin .dockerignore), o quede expuesto en logs de CI/CD. No es apto para producción porque no hay control de acceso ni auditoría sobre quién lee el secreto.

Variable de entorno del shell

Se exporta el secreto en la sesión del terminal antes de lanzar el contenedor:

```
export HF_TOKEN=hf_xxx
docker compose up
```

Docker Compose la recoge automáticamente si está referenciada como \${HF_TOKEN} en el yml. No hay fichero en disco, pero el secreto queda visible en ps aux, en el historial del shell y en los metadatos del contenedor (docker inspect). Útil en pipelines de CI/CD (GitHub Actions, GitLab CI) donde el secreto se inyecta como variable de entorno protegida por la plataforma y nunca toca el disco.

Docker Secrets (modo Swarm)

Docker Swarm permite almacenar secretos cifrados en el plano de control del cluster. El secreto se monta como fichero en /run/secrets/<nombre> dentro del contenedor, nunca como variable de entorno. Solo los servicios autorizados pueden acceder a él y está cifrado en tránsito y en reposo. Indicado para producción con Docker Swarm, aunque su uso está decayendo frente a Kubernetes + Secrets o soluciones externas.

Servicios externos (AWS Secrets Manager, HashiCorp Vault, etc.)

El secreto nunca sale del servicio gestor en texto plano. La aplicación lo solicita en tiempo de ejecución mediante una llamada autenticada a la API, con control de acceso granular, rotación automática y auditoría completa de cada acceso. Requiere que el contenedor tenga identidad (IAM role, Vault token) para autenticarse. Es el estándar en producción cloud (AWS, GCP, Azure) o en entornos empresariales con Vault, donde la seguridad y la trazabilidad son requisitos.

Resumen de elección

- Desarrollo local controlado: .env + .gitignore
- Desarrollo local / CI efímero: variable de entorno del shell
- Producción con Docker Swarm: Docker Secrets
- Producción cloud / enterprise: AWS Secrets Manager, Vault

RAZONAMIENTO — Health check y arranque ordenado

Análisis de escenarios

Escenario 1: modelo pequeño cargado localmente (< 1 segundo)

El backend arranca casi instantáneamente. Con un depends_on simple el frontend probablemente llegará cuando el modelo ya está en memoria, por lo que en la práctica no se producen errores. El problema es latente pero no observable: si el host está bajo carga o el scheduler del SO retrasa el proceso, el frontend puede ganar la carrera. La solución con health check es igualmente correcta pero el margen de fallo es muy pequeño.

Escenario 2: modelo descargado desde Hugging Face Hub (latencia variable)

Este es el caso del proyecto. La descarga puede tardar desde unos pocos segundos hasta más de un minuto dependiendo de la conexión y del tamaño del modelo. Con `depends_on` simple, el frontend arranca en cuanto el proceso de `uvicorn` está en pie (menos de 1s), mucho antes de que el modelo esté en memoria. Cualquier petición de predicción durante ese periodo recibe HTTP 503. Si el frontend hace una llamada al arrancar (precalentamiento, comprobación de conectividad, etc.) fallará y puede dejar la interfaz en estado de error permanente. El health check con `condition: service_healthy` resuelve esto completamente.

Escenario 3: modelo grande con compilación/warmup (decenas de segundos)

La ventana de fallo es amplia y garantizada. Sin health check el frontend arranca con certeza antes de que el backend esté listo. Además, el `--start-period` del HEALTHCHECK cobra especial importancia: debe ser suficientemente largo para que Docker no marque el backend como unhealthy durante la fase normal de carga. Si `start-period` es demasiado corto, Docker podría reiniciar el backend antes de que termine de cargar el modelo, creando un bucle de reinicios. En este escenario también conviene aumentar `--retries` para tolerar variabilidad en los tiempos de warmup.

Pseudocódigo de la solución

```
# Dockerfile del backend
HEALTHCHECK --interval=10s --timeout=5s --start-period=30s --retries=3
  CMD curl -f http://localhost:8000/health || exit 1

# docker-compose.yml
frontend:
  depends_on:
    backend:
      condition: service_healthy # espera a que /health devuelva 200

# backend/main.py — endpoint /health
GET /health:
  si model_state["loaded"] == True:
    devolver HTTP 200 { status: "ok", model_loaded: true, ... }
  si model_state["load_error"] != None:
    devolver HTTP 503 { status: "error", model_loaded: false, ... }
  en otro caso:
    devolver HTTP 503 { status: "not_ready", model_loaded: false, ... }
```

El flujo resultante es:

1. Docker levanta el backend.
2. Cada 10s Docker llama a GET /health. Mientras el modelo se descarga, recibe 503.
3. En cuanto el modelo está en memoria, /health devuelve 200 -> el backend pasa a healthy.
4. Solo entonces Docker arranca el frontend.
5. El frontend puede hacer peticiones al backend con garantía de que el modelo está listo.